

CHAPTER 3

SYSTEM DEVELOPMENT METHODOLOGY

3.1 Introduction

This chapter details the design and development procedure of the proposed PDMS using cloud computing and DevSecOps approach and Justifies the methodology against the project requirements. The design and deployment of a secure cloud infrastructure shows a distinct set of procedural challenges: the system must be developed in iterations to allow the security requirements to be validated on each stage, the pipeline should integrate security tools that execute after every code change, and the infrastructure will follow an incremental build, test, and hardening without requiring a complete rebuild between phases.

Section 3.2 presents the chosen methodology, An Agile development integrated with DevSecOps practices and justifies this choice. Section 3.3 describes the phases of the methodology as applied to the proposed project with each phase and its deliverables. Section 3.4 provides a technical walkthrough of each tool and technology used in the system. Section 3.5 presents the complete system requirement analysis, covering both functional and non-functional requirements. Section 3.6 summarises the chapter.

3.2 Methodology Choice and Justification

This section presents the development methodology selected for this project and provides a structured justification for that choice. A candidate methodology were evaluated Agile with DevSecOps integration against the specific requirements of a cloud security system development project.

3.2.1 Overview of Candidate Methodology

Agile with DevSecOps integration expands on Scrum by introducing security testing, scanning for vulnerabilities, and compliance into the sprint cycle and CI/CD

pipeline. In other words, instead of performing security tests at some phase following the implementation, they are integrated directly into the process itself such that the product increments have inherent security properties. DevSecOps integration into the Agile cycle is referred to as a natural extension of Agile to the sphere of security, where, just as Agile blurred the lines between the phases of requirements gathering and implementation, DevSecOps does the same to development and security. This principle has been formulated early on in DevSecOps research (Bass *et al.*, 2015), confirmed by more recent research on resilient cloud deployments Paidy and Chaganti (2024). Paidy and Chaganti (2024)

3.2.2 Justification for Agile with DevSecOps

The Agile with DevSecOps methodology is selected for this project on the basis of four aligned criteria.

Iterative security validation. The system's security architecture, VPC network topology, IAM policy structure, encryption configuration, and CI/CD pipeline should be tested at every step of building process. It is not sufficient to test it once. Due to the agile nature of sprints, every piece of infrastructure can be constructed, analyzed, and verified before another piece is relying on its work.

Shift-left security integration. The early execution of security tests in DevSecOps by ensuring that security is integrated as early as possible in the software development process.hence identifying vulnerabilities during commit and not deployment when it becomes expensive to fix.

Infrastructure as Code compatibility. Since infrastructure is declared in Terraform, it fits into Agile development methodology very well. This means that at each sprint, developers will have a version-controlled configuration for their environment. It would be possible to rebuild the entire environment starting with any version of the code and even roll it back to an earlier version if necessary.

Measurable compliance. The HIPAA compliance goal demands that security postures are constantly measured rather than being evaluated only once at the end of the development process.

Table 3.1 summarises the comparison of the three candidate methodologies against the project's key requirements. Figure 3.1 Shows the DevSecOpd Sprint cycle.

Table 3.1 Methodology Comparison

Criteria	Waterfall	Scrum (Agile)	Agile + DevSecOps
Iterative development	×	✓	✓
Security integrated at each stage	×	×	✓
Supports IaC incremental build	Partial	✓	✓
Automated pipeline compatibility	×	Partial	✓
Continuous compliance measurement	×	×	✓
Suited for changing security requirements	×	✓	✓

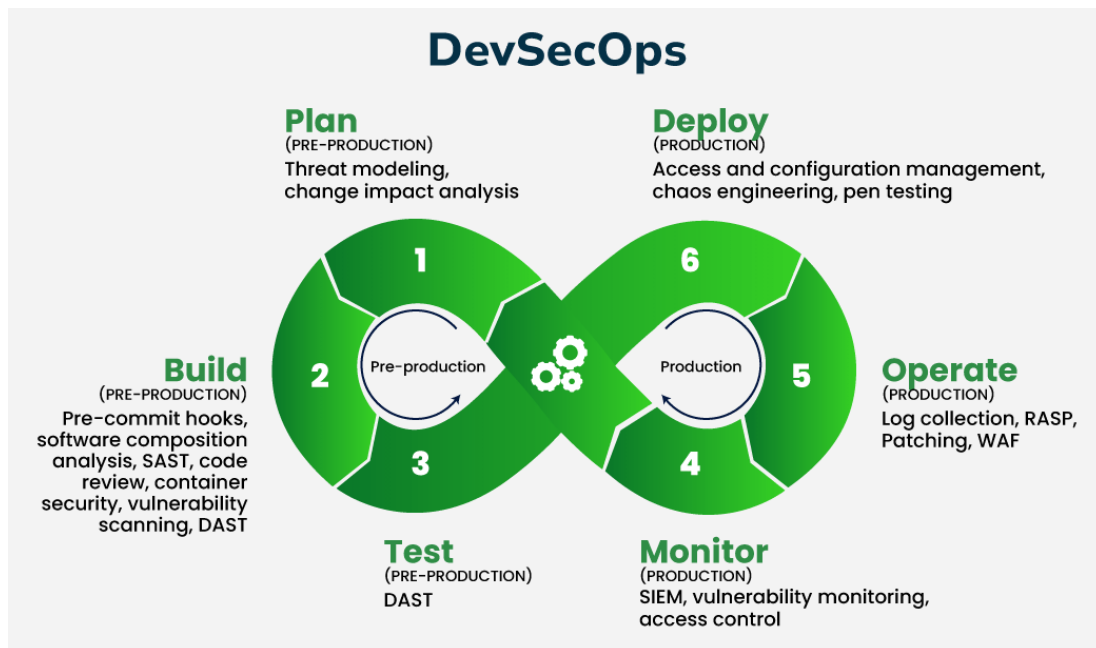


Figure 3.1 Agile + DevSecOps Sprint Cycle with Integrated Security Gates

3.3 Phases of the Chosen Methodology

The project is organized into five sprints. Each sprint has a defined scope, a set of deliverables, and a security gate that must pass before the sprint is considered complete. The full project schedule spanning all five sprints is presented in Appendix A, Figure A.2.

3.3.1 Sprint 1: Requirements and Architecture Design

Scope: Determine and document both functional and non-functional requirements of the system. System Architecture Design should be produced along with VPC network architecture, IAM policies and DevSecOps pipeline design.

Deliverables: Document of system requirements (Chapter 3.5), VPC network architecture, IAM policies and CI/CD pipeline.

Security gate: Perform architectural review for the technical safeguard requirements under the HIPAA Security Rule. All control gaps shall be identified and fixed in the design phase before moving to Sprint 2.

3.3.2 Sprint 2: Network Infrastructure and Database Layer

Scope: Provision the AWS VPC using Terraform, including the public subnet (Application Load Balancer), private application subnet (EC2), and isolated database subnet (RDS). Configure Security Groups, Network ACLs, NAT Gateway, and Internet Gateway. Deploy and configure the Amazon RDS database instance with KMS encryption at rest.

Deliverables: Terraform configuration files for VPC, subnets, routing tables, security groups, and RDS instance. Checkov scan report for all IaC configurations.

Security gate: Checkov scan must report zero critical misconfigurations. RDS instance must be confirmed as unreachable from the public internet. Encryption at rest must be verified in the AWS Console.

3.3.3 Sprint 3: Application Layer and Authentication

Scope: Deploy the Docker-containerised application that consist of React frontend and Node.js/Express backend to EC2 instances in the private application subnet. Implement the RBAC authentication system with three roles (Doctor, Admin, Patient). Configure the Application Load Balancer with HTTPS termination and TLS certificate.

Deliverables: Dockerfiles for frontend and backend, application deployment Terraform modules, IAM role definitions, Trivy scan report for container images.

Security gate: Trivy scan must report zero critical CVEs in deployed container images. TLS termination must be verified at the ALB. Application-level RBAC must be confirmed to enforce role boundaries through test cases covering each user role.

3.3.4 Sprint 4: DevSecOps Pipeline and Monitoring

Scope: Design and set up the CI/CD pipeline on GitHub Actions, using Trivy, SonarQube, and Checkov as automated steps within the pipeline. Set up the Amazon CloudWatch log groups and alarm metrics. Set up the AWS CloudTrail audit logs for all the API calls and access events.

Deliverables: GitHub Actions pipeline YAML files, SonarQube settings, CloudWatch dashboard settings, CloudTrail settings.

Security gate: The pipeline should show that any commit that is injected with a critical vulnerability (test CVE injection) gets blocked from proceeding beyond the build phase and does not reach the deployment stage. CloudTrail should have been shown to track access to patient data.

3.3.5 Sprint 5: Security Evaluation and Compliance Testing

Scope: Implement the entire security assessment framework, which includes automated vulnerability assessments from Trivy, SonarQube, and Checkov; black box and white box pen testing of the application; Recovery Time Objective (RTO) test of a simulated ransomware wipe and redeploy attack; and an AWS Security Hub HIPAA posture assessment.

Deliverables: Security assessment report; pen testing report; RTO test report that captures the time taken to recover; and AWS Security Hub HIPAA posture assessment report.

Security gate: RTO test is done and documented. The HIPAA posture score obtained from AWS Security Hub should be documented as the key measure for compliance. The critical findings from Security Hub should be remediated before sprint completion.

3.4 Technology Used Description

this section. highlight each tool and service used in the proposed system

3.4.1 Cloud Infrastructure: Amazon Web Services (AWS)

Amazon Web Services is the cloud provider for the proposed system. below are AWS services that's used:

Amazon Virtual Private Cloud (VPC) it enable creating an Isolated of logical network in the AWS cloud environment is made possible by this strategy. The proposed

design suggests the creation of one VPC that includes three subnets, namely the public subnet, the application subnet, and the database subnet. The subnets should span across several Availability Zones. VPCs are the main tool to create network isolation in Alamin Clinic.

Amazon EC2 (Elastic Compute Cloud) it servers as the backend server which the application will run on. EC2 instances are deployed within the private application subnet, accessible only through the Application Load Balancer, and are not directly reachable from the internet.

Amazon RDS (Relational Database Service)The managed relational database is provided by RDS. The service is installed in the private subnet without internet access. Data at rest encryption is done by KMS, and backups have been set up for point-in-time recovery. The system utilizes Amazon RDS for PostgreSQL which is running in the private subnet with KMS at rest encryption.

Application Load Balancer (ALB) distributes incoming HTTPS traffic across EC2 instances. The ALB terminates TLS, ensuring that all traffic entering the application layer is encrypted in transit. It is the only entry point to the application from the public internet.

AWS Key Management Service (KMS) provides managed cryptographic keys for encrypting the RDS database and any sensitive data stored in Amazon S3. AES-256 encryption is applied at rest.

Amazon CloudWatch Monitors and alerts on the infrastructure being deployed. The log group captures application logs and system metrics; alarms are set up for alerting on any irregular behavior in patterns like multiple failed logins or API calls.

AWS CloudTrail records every AWS API call within the account. This satisfies the HIPAA requirement for audit logging and provides logs capability to help in a security incident.

AWS Security Hub Aggregates security data from various services in AWS and measures the overall security stance against the HIPAA Security Rule. Security Hub is the main compliance metric tool used in this project.

3.4.2 Infrastructure as Code: Terraform

Terraform, an IaC software created by HashiCorp, is used as the Infrastructure as Code software to create and manage all AWS services in this project. Terraform uses HCL, a declarative configuration language, where the final configuration of the infrastructure to be created is declared and then the order in which API requests are to be made is decided.

3.4.3 Containerisation: Docker

Containerization using docker is utilized for packaging the application frontend and backend components as Docker images. Containerization ensures consistency in the application runtime environment during the development, testing, and production phases, which helps avoid any configuration mismatch across these different phases. The Docker images are created through the CI/CD pipeline and then scanned using Trivy before pushing to the container registry.

3.4.4 CI/CD Pipeline: GitHub Actions

GitHub Actions acts as the CI/CD platform utilized for automating the build, scanning, and deployment process pipeline. Workflow set up via YAML gets triggered on each commit pushed into the main branch of the repository. Following this, the pipeline goes through the below-stated processes in order:

- (a) Code checkout and dependency installation
- (b) SonarQube SAST scan of application source code
- (c) Trivy container image vulnerability scan
- (d) Checkov IaC misconfiguration scan of Terraform files
- (e) Terraform plan and apply (deployment) only if all scan stages pass

A failure at any scan stage with a critical or high-severity finding terminates the pipeline and blocks deployment.

3.4.5 Security Scanning Tools

Trivy (Aqua Security) is an open source software vulnerability scanner that supports container images, file system, and Git repository scanning. Trivy scans Docker images against the National Vulnerability Database (NVD) in order to identify known CVEs in the base image and its dependencies.

SonarQube is an example of Static Application Security Testing (SAST). This testing tool performs static code analysis to find security flaws in the code, code quality problems, as well as injection vulnerabilities.

Checkov is an infrastructure as code static analysis tool that evaluates Terraform configurations against security and compliance policy knowledge bases. In this particular implementation, the following policies among others are enforced using Checkov: Encryption should be turned on for all RDS instances; S3 buckets should not be accessible to the public; No Security Groups should allow unrestricted inbound SSH; and CloudTrail should be enabled.

3.4.6 Application Stack

The **React** framework powers the JavaScript components that make up the web frontend of the system, including the patient portal, the appointment scheduling UI, and the administrative dashboards. The React app is served as a static package using the Application Load Balancer. Figure 3.2 Shows the application stack.

Node.js with Express makes up the JavaScript runtime and web framework used in the backend of the application. This gives us the RESTful API layer that handles authentication, role authorization, and data access.

PostgreSQL This is the relational database management system running on Amazon RDS. The PostgreSQL database system was selected because of its excellent

capabilities in enforcing row-level security policies that enhance the role-based access control system through database-level restrictions per each user role.

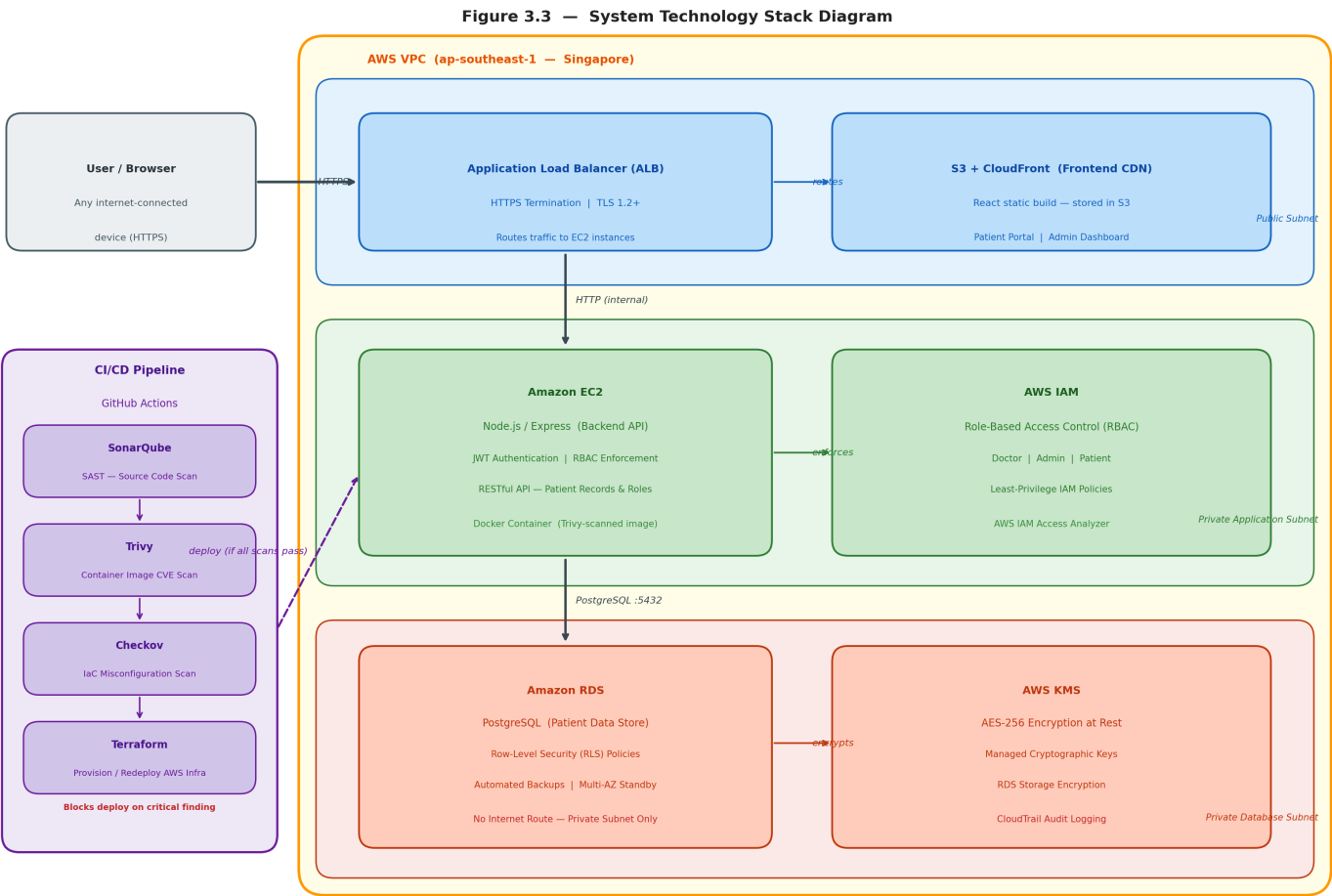


Figure 3.2 System Technology Stack Diagram

3.5 System Requirement Analysis

System requirements have been divided into two groups: functional requirements which define what the system will do, and non-functional requirements which set the constraints for quality within which the system should work. These two groups of requirements have been taken straight from the problems found in the case study of Alamin Clinic (Chapter 2) and HIPAA Security Rule technical safeguards requirements.

3.5.1 Functional Requirements

Functional requirements define the operations and behaviors that need to be supported by the system. The system consists of twelve functional requirements categorized into five areas such as user login and management (FR-07, FR-08, FR-10), patient registration and management (FR-01), medical record management (FR-02, FR-03, FR-06), scheduling of appointments (FR-04, FR-05), and security controls (FR-09, FR-11, FR-12). Table C.1 in the appendix C provides all functional requirements.

3.5.2 Non-Functional Requirements

Non-functional requirements define the performance, security, reliability, and compliance constraints the system must satisfy. The system has eleven non-functional requirements across five categories: security (NFR-01 through NFR-04: KMS encryption, TLS, least-privilege IAM, pipeline blocking), availability and recovery (NFR-05: 99.9% uptime; NFR-06: RTO 15 minutes), compliance and auditability (NFR-07: HIPAA Security Hub posture; NFR-08: CloudTrail 90-day retention), performance (NFR-09: API response 3 s under 50 concurrent users), scalability (NFR-10: EC2 Auto Scaling), and maintainability (NFR-11: all infrastructure in Terraform). The complete requirements table is provided in Appendix C, Table C.2.

3.6 Chapter Summary

This chapter defined the Agile with DevSecOps methodology adopted for the project and justified its selection over Waterfall and standard Scrum approaches on the basis of four criteria: iterative security validation, shift-left pipeline integration, Infrastructure as Code compatibility, and measurable compliance tracking.

The project is structured into five sprints. Sprint 1 covers requirements and architecture design. Sprints 2 through 5 cover network infrastructure, application layer, pipeline integration, and security evaluation respectively. Each sprint is bounded by a security gate that must be cleared before the next sprint proceeds.

The technology stack was described across six categories: AWS cloud services (VPC, EC2, RDS, ALB, KMS, CloudWatch, CloudTrail, Security Hub), Terraform IaC, Docker containerisation, GitHub Actions CI/CD, security scanning tools (Trivy, SonarQube, Checkov), and the application stack (React, Node.js/Express, PostgreSQL). Each technology selection was grounded in the security and operational requirements established in the literature review.

The system requirement analysis produced twelve functional requirements and eleven non-functional requirements, each with a defined verification method. The non-functional requirements establish the measurable security, availability, recovery, and compliance targets against which the system will be evaluated in Chapter 5. Chapter 4 proceeds to translate these requirements into the complete system design.